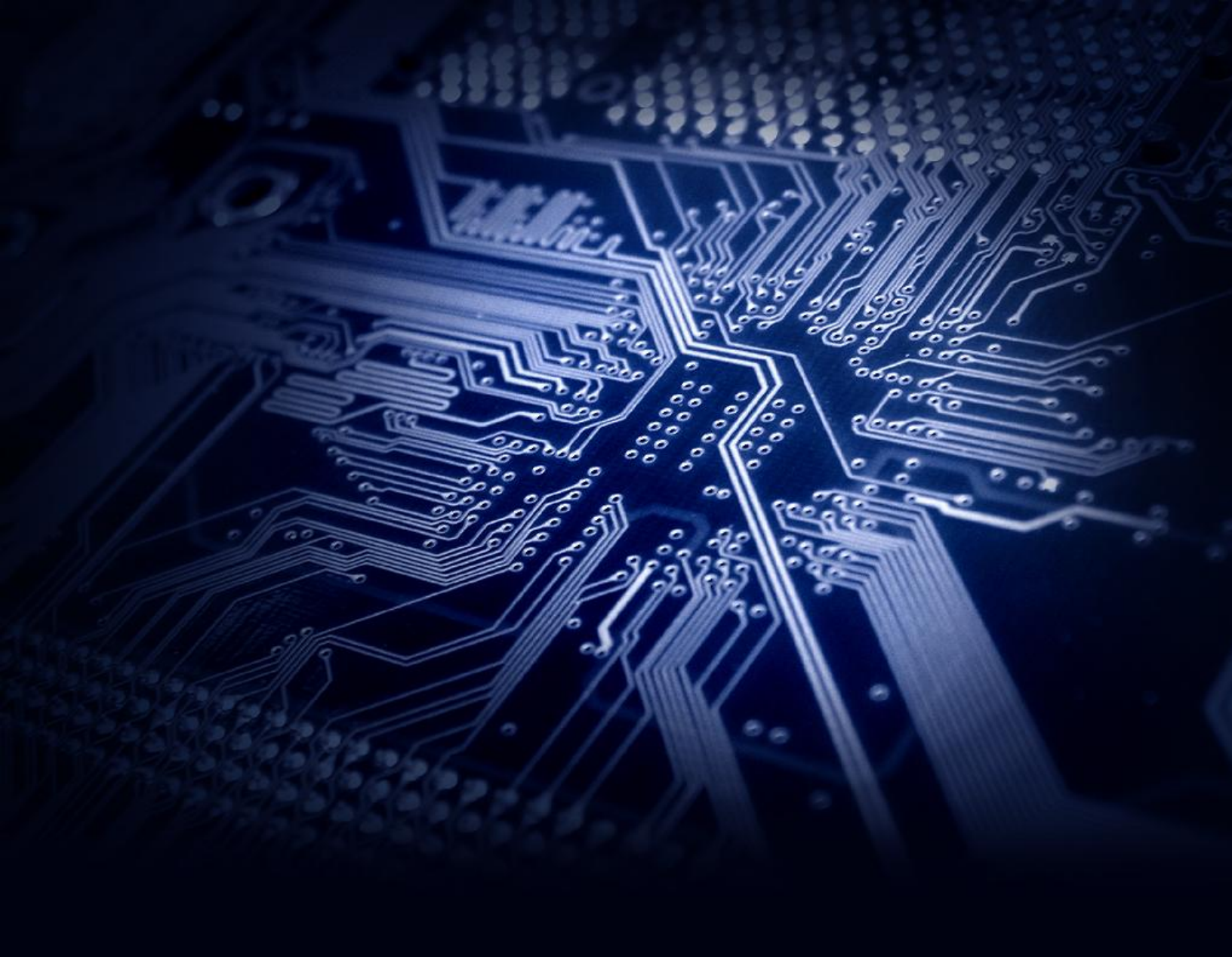




SPI Protocol with APB Interface

Digital Design Using FPGA

RTL Modules: APB interface / SPI Master / SPI Slaves

Prepared by:

Tarek Ahmed
George Ashraf
Amir Sherif

This project implements a memory-mapped SPI (Serial Peripheral Interface) controller that is accessed by a processor or bus master through the APB (Advanced Peripheral Bus) protocol, one of the buses defined in ARM's AMBA specification. The design is split into three RTL modules that together form a complete master-side SPI subsystem plus a reference SPI slave for verification and demonstration:

- Conceptually, the block sits between a parallel, register-based world (the APB bus, which a CPU or DMA engine can read and write) and a serial, pin-based world (SPI, used by many external devices such as flash memories, ADCs/DACs, sensors, and display controllers). The APB–SPI interface is the translator between the two.



SPI is a synchronous, full-duplex, point-to-point serial protocol. A master generates the clock (SCLK) and selects a target device with a chip-select line (CS), while data is shifted simultaneously in both directions on two data lines — MOSI (master out, slave in) and MISO (master in, slave out). Because the same clock edge that shifts a bit out of the master shifts a bit into the slave, SPI achieves high throughput with a very simple, small-footprint interface, at the cost of needing a dedicated chip-select per slave device.

1.2 What is APB?

APB is the low-complexity member of the AMBA bus family, intended for register-style access to peripherals that do not need high bandwidth. A transfer takes two bus cycles: a SETUP phase (PSEL asserted) followed by an ACCESS phase (PENABLE asserted), during which PWDATA/PADDR/PWRITE select the operation and the slave returns PRDATA and (optionally) inserts wait states via PREADY. It is simple to decode and well suited to control/status register blocks like the one in this project.

2. How the Design Works

2.1 APB–SPI Interface — Register Map

The `apb_spi_interface` module decodes a 4-bit PADDR into four word-aligned registers. Writes are captured synchronously on PCLK when PSEL, PENABLE and PWRITE are all asserted; reads are combinational, so the module returns data in the same access phase with PREADY tied permanently high (a zero-wait-state slave). PSLVERR is tied low, meaning the current version does not flag any error responses.

Address	Register	Access	Description
0x0	TX_DATA	W	32-bit data to transmit. Loaded into the SPI master's shift register when a transaction starts.
0x4	RX_DATA	R	32-bit data most recently received from the SPI slave (master's <code>data_out</code>).
0x8	CTRL	R/W	Control bits — see field table below.
0xC	STATUS	R	bit 0 = BUSY, bit 1 = DONE (mirrors the master's status flags).

CTRL register (0x8) bit fields:

Bit(s)	Field	Meaning
0	START	One-clock pulse that tells the SPI master to begin a new transaction.
1	CPOL	Clock polarity — idle level of SCLK.
2	CPHA	Clock phase — which edge data is sampled/shifted on.
4:3	SLAVE_SEL	Selects which of up to 4 chip-select lines is driven active.

Bit(s)	Field	Meaning
8:5	DVSR	4-bit clock divider that sets the SPI clock frequency from the system clock.

2.2 SPI Master — Operation

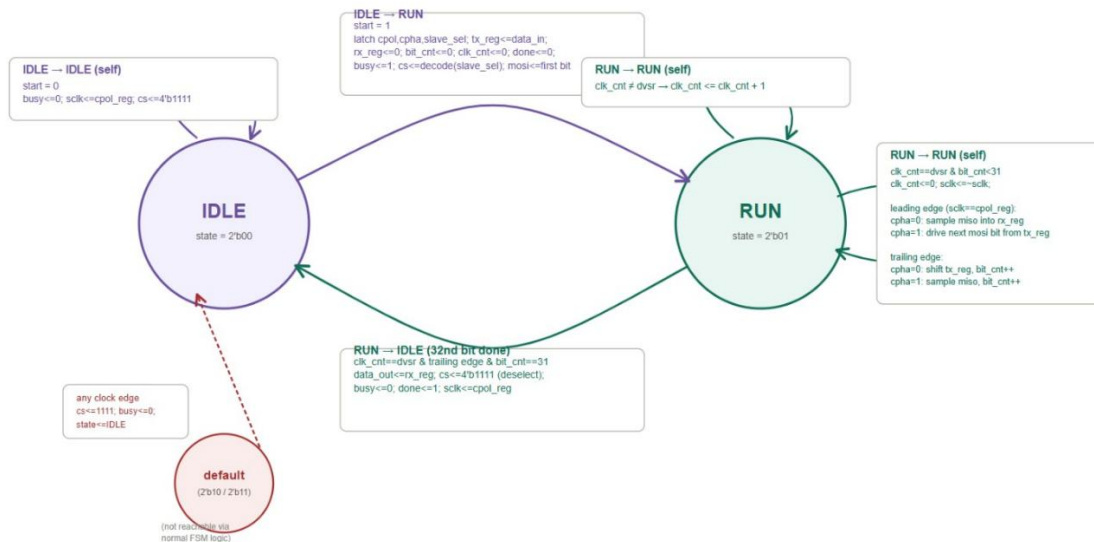
master_spi_final_form is built around a two-state FSM (IDLE and RUN):

- IDLE: the master holds SCLK at the idle level defined by CPOL and de-asserts all chip-selects. When it sees the START pulse, it latches CPOL, CPHA and SLAVE_SEL into internal registers, loads the 32-bit data_in into the transmit shift register, clears the receive register and bit counter, drives the selected chip-select active (one-cold: exactly one of 4 bits low), and moves to RUN.
- RUN: a 4-bit counter (clk_cnt) compares against the programmed divider (dvsr) to generate SCLK edges from the system clock, so the SPI clock frequency is software-configurable. On each SCLK edge the design checks CPHA to decide whether that edge is a sample edge (shift a MISO bit into the receive register) or a shift edge (drive the next MOSI bit out), correctly reproducing all four standard SPI clock modes. This repeats for 32 bits; on the final bit the received word is latched into data_out, the chip-select is released, SCLK returns to its idle level, BUSY is cleared, DONE is pulsed, and the FSM returns to IDLE.

The busy/done pair lets software (through the STATUS register) know when it is safe to write new TX_DATA or read a fresh RX_DATA, without needing an interrupt line.

master.v — Internal FSM (the `state` register)

2-bit state register: state = IDLE (2'b00) or RUN (2'b01). Labels show: condition / key actions on that edge.



2.3 SPI Slave — Operation

The slave module is a minimal reference implementation used to close the loop with the master for testing: while its chip-select is active, it shifts MOSI into a 32-bit receive register on the rising edge of SCLK, and on the falling edge of SCLK it shifts the next bit of that received word back out on MISO and advances a bit counter, asserting DONE once 32 bits have been exchanged.

2.4 Typical Transaction Sequence

- 1. The APB master (CPU) writes the word to transmit to TX_DATA (0x0).
- 2. The APB master writes CTRL (0x8) with the desired CPOL, CPHA, SLAVE_SEL and DVSR, and START = 1 in the same write.
- 3. The APB–SPI interface forwards these fields directly to the SPI master; START produces a single clock pulse that kicks off the FSM.
- 4. The SPI master asserts the chosen chip-select and shifts 32 bits in and out over SCLK/MOSI/MISO, generating the bus clock itself from DVSR.
- 5. Software polls STATUS (0xC) until BUSY = 0 and DONE = 1, then reads the received word from RX_DATA (0x4).

3. Problem the Project Solves

Many peripherals a system needs to talk to — flash memory, ADCs/DACs, sensors, display drivers — use SPI because it is cheap, pin-efficient and fast enough for these use cases. However, the processor side of a typical SoC communicates over a parallel, register-addressed bus, not directly over serial pins. Two problems follow from this mismatch, and this project addresses both:

- Protocol mismatch: SPI has no concept of addressable registers, wait states or bus arbitration, while the CPU only understands memory-mapped reads and writes. The `apb_spi_interface` module is exactly the bridge that resolves this — it turns SPI configuration and data movement into ordinary register accesses (TX_DATA, RX_DATA, CTRL, STATUS) that any APB-capable master can perform with normal load/store instructions.
- Manual, CPU-driven bit-banging: without a hardware SPI controller, software would have to toggle SCLK/MOSI and sample MISO itself, consuming CPU cycles for every single bit and making precise timing (especially clock-mode-dependent sampling) difficult to guarantee. Moving the shifting, clock generation and mode handling into the `master_spi_final_fsm` FSM frees the processor to simply write data and poll a status bit, while the hardware guarantees correct SPI timing.

In short, the project turns a low-level, timing-sensitive serial protocol into a small set of memory-mapped registers that software can drive safely and efficiently, which is precisely the role an SPI controller peripheral plays in a real SoC.

4. Advantages of This Design

- Standard-bus compliant: implements the APB protocol correctly (PSEL/PENABLE/PWRITE handshake, PREADY, PSLVERR), so it can be dropped into any AMBA-based SoC alongside other APB peripherals without custom glue logic.
- Full SPI mode coverage: CPOL and CPHA are both software-programmable per transaction, so the same hardware can talk to slaves that expect any of the four standard SPI modes (0–3), rather than being hard-wired to one.
- Software-configurable clock rate: the 4-bit DVSR field lets software trade off SPI throughput versus signal integrity/EMI per target device, without changing hardware.
- Multi-slave support: a 2-bit SLAVE_SEL field and a one-cold, active-low 4-bit chip-select bus let a single master address up to four independent SPI slaves from one controller instance.

- Zero-wait-state register access: PREADY is tied high, so every APB register access completes in the minimum two cycles, keeping register access latency low and predictable.
- Simple, non-blocking software model: the BUSY/DONE pair in STATUS lets firmware poll for completion (or be extended to drive an interrupt) instead of blocking on the SPI transfer, and TX/RX are decoupled into separate registers so a new transaction can be queued as soon as the previous one is read.
- Modular, reusable RTL: the APB register file, SPI master and SPI slave are separate modules with clean, narrow interfaces (start/busy/done, data_in/data_out), so each can be simulated, verified and reused independently — e.g. the master can be reused with a different bus wrapper (AHB, Wishbone, etc.) with no changes to its internals.
- Small footprint: the whole controller is built from a handful of registers, counters and combinational decode logic — no memory, no complex arbitration — which keeps area and power low, well suited to embedded and low-power SoCs.

5. Design Notes and Possible Extensions

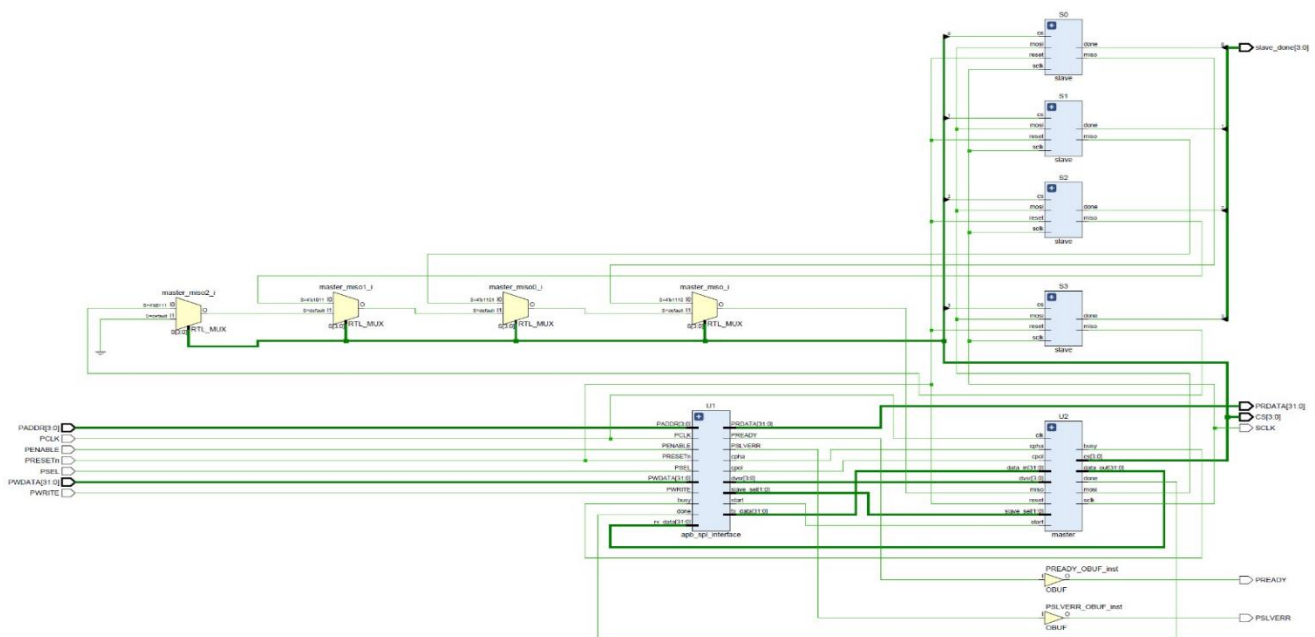
A few characteristics of the current RTL are worth flagging for anyone building on this project further — not as defects, but as known scope boundaries of this version:

- No error reporting: PSLVERR is hard-wired to 0, so malformed APB accesses cannot be signalled back to the bus master.
- Polling only: completion is only observable through the STATUS register; there is no interrupt output, so the CPU must poll BUSY/DONE rather than being woken on completion.
- Fixed 32-bit transfer length: both the master and the reference slave always exchange exactly 32 bits per transaction; variable-length transfers (e.g. 8/16-bit) would need an additional length field.
- Divider range: DVSR is 4 bits wide, limiting the number of selectable SPI clock rates from a given system clock to 16 steps.
- Reference slave is a template: the slave module is intentionally minimal (no APB side of its own) and is best understood as a loopback/verification target for the master rather than a production-ready slave peripheral.

6. Conclusion

Together, the three modules form a compact, standards-compliant SPI master peripheral that a processor can drive entirely through four memory-mapped registers over APB. The design cleanly separates bus-facing register logic (apb_spi_interface) from protocol-facing serial logic (master_spi_final_form), supports all four SPI clock modes and multiple slaves, and gives software a simple, non-blocking programming model — making it a practical building block for any AMBA-based SoC that needs to talk to SPI peripherals.

7. Synthesis diagram



8. Test bench walk through

1. APB-SPI Interface Testbench

This testbench is a register-level test for `apb_spi_interface` and it performs an APB write or read to every address in the map and checks the resulting internal signal or `PRDATA` value against what the register map defines.

1.1 Environment Setup

```
reg          PCLK;
reg          PRESETn;
reg          PSEL;
reg          PENABLE;
reg          PWRITE;
reg  [3:0]    PADDR;
reg  [31:0]   PWDATA;
wire [31:0]   PRDATA;
wire          PREADY;
wire          PSLVERR;

wire [7:0]    CTRL;
wire [7:0]    TXDATA;
reg  [7:0]    RXDATA;
reg  [7:0]    STATUS;

apb_spi_interface DUT ( .* );
```

The APB-side signals (PSEL, PENABLE, PWRITE, PADDR, PWDATA) are driven by the testbench as reg, exactly as a real APB master would drive them.

1.2 Clock Generation

```
initial begin
    PCLK = 0;
    forever #5 PCLK = ~PCLK;
end
```

A free-running clock toggles every 5 ns, giving a 10 ns period (100 MHz). Every task and check in this testbench is timed relative to this clock.

1.3 The apb_write Task

```
task apb_write;
    input [3:0]  addr;
    input [31:0] data;
    begin
        #10;
        PSEL     = 1;
        PENABLE  = 0;
        PWRITE   = 1;
        PADDR    = addr;
        PWDATA   = data;

        #10;
        PENABLE  = 1;

        #10;
        PSEL     = 0;
        PENABLE  = 0;
        PWRITE   = 0;
        PADDR    = 0;
        PWDATA   = 0;
        #1;
    end
endtask
```

This task reproduces a correct two-phase APB write exactly as the protocol defines it: SETUP phase (PSEL asserted, PENABLE low, address/data/PWRITE driven), then ACCESS phase one clock later (PENABLE raised while PSEL and the address/data hold steady), then the bus is returned to idle. Because apb_spi_interface only registers PWDATA when PSEL && PENABLE && PWRITE are all high, this task is what allows every later write to land in the correct register on the correct clock edge.

What to expect:

Calling apb_write(addr, data) should, one clock after the ACCESS phase, update whichever internal register that address maps to (tx_data at 0x0, the CTRL fields at 0x8).

1.4 The apb_read Task

```
task apb_read;
    input [3:0]  addr;
    input [31:0] expected;
    begin
        #10;
        PSEL      = 1;
        PENABLE    = 0;
        PWRITE     = 0;
        PADDR      = addr;
        PWDATA     = 0;

        #10;
        PENABLE    = 1;
        #1;

        if (PRDATA == expected)
            $display("READ PASS: ADDR=%h DATA=%h", addr, PRDATA);
        else
            $display("READ FAIL: ADDR=%h EXPECTED=%h GOT=%h", addr, expected, PRDATA);

        #9;
        PSEL      = 0;
        PENABLE    = 0;
        PADDR      = 0;
    end
endtask
```

Same two-phase APB timing as `apb_write`, but with `PWRITE` held low. One nanosecond after `PENABLE` goes high giving the combinational read-decode logic in the DUT time to settle, the task compares `PRDATA` against an expected value passed in by the caller and immediately prints a PASS or FAIL line. This is the self-checking mechanism used for every register read in the rest of the test.

What to expect:

Every call to `apb_read` prints exactly one line: “READ PASS: ADDR=... DATA=...” if `PRDATA` matched, or “READ FAIL: ADDR=... EXPECTED=... GOT=...” if it didn't.

1.5 Reset Test

```
PRESETn = 0;
#12;
PRESETn = 1;
#10;

if (tx_data == 0 && start == 0 && cpol == 0 &&
    cpha == 0 && slave_sel == 0 && dvsr == 0)
    $display("RESET PASS");
else
    $display("RESET FAIL");
```

PRESETn is held low for 12 ns (more than one full clock period) and then released. The check confirms every register the DUT's reset block touches tx_data, start, cpol, cpha, slave_sel, dvsr has returned to its all-zero default.

What to expect:

“RESET PASS”. If any control register failed to clear, you would see “RESET FAIL” instead, and you'd know to check the DUT's asynchronous reset block for a missing signal.

1.6 PREADY and PSLVERR Checks

```
if (PREADY == 1)
    $display("PREADY PASS");
else
    $display("PREADY FAIL");

if (PSLVERR == 0)
    $display("PSLVERR PASS");
else
    $display("PSLVERR FAIL");
```

These simply confirm the two static APB response signals are wired the way the design intends: PREADY permanently high (no wait states) and PSLVERR permanently low (no error responses implemented) as we said in the presentation its an edge here for our design because every register access completes in 2 cycles.

What to expect:

“PREADY PASS” and “PSLVERR PASS”, unconditionally these two signals are tied combinationaly in the DUT, so they cannot depend on any prior stimulus.

1.7 TX_DATA Write

```
apb_write(4'h0, 32'h1234_ABCD);

if (tx_data == 32'h1234_ABCD)
    $display("TX_DATA WRITE PASS");
else
    $display("TX_DATA WRITE FAIL");
```

A straightforward write to address 0x0 followed by a direct check that the tx_data output register now holds the written value.

What to expect:

“TX_DATA WRITE PASS”, with tx_data reading back 32'h1234ABCD on the waveform from this point until it's overwritten.

1.8 CTRL Register Write and Read

```
apb_write(4'h8, 32'hE);

if (cpol == 1 && cpha == 1 &&
    slave_sel == 1 && dvsr == 0)
    $display("CTRL CONFIG PASS");
else
```

```
$display("CTRL CONFIG FAIL");
```

```
apb_read(4'h8, 32'hE);
```

32'hE is binary 000000000000000000000000**00001110** I bolded the required data for CTRL

Decoded against the CTRL bit map: bit 0 (START) = 0, bit 1 (CPOL) = 1, bit 2 (CPHA) = 1, bits [4:3] (SLAVE_SEL) = 01 = 1, bits [8:5] (DVSR) = 0000 = 0. The check confirms the DUT split these fields out correctly into separate internal registers, and the subsequent apb_read confirms the DUT can also reassemble them correctly back into a single 32-bit word for a CTRL readback.

What to expect:

“CTRL CONFIG PASS” followed by “READ PASS: ADDR=8 DATA=e”. Together these prove the CTRL register is symmetric what you write is what you read back (aside from the START bit, which self clears and is always read back as 0).

1.9 START Pulse Behavior

```
PSEL = 1; PENABLE = 0; PWRITE = 1;
PADDR = 8; PWDATA = 1;
#10;
PENABLE = 1;
#10; // wait for the write clock edge

if (start == 1)
    $display("START PASS");
else
    $display("START FAIL: start = %b", start);

PSEL = 0; PENABLE = 0; PWRITE = 0;
PADDR = 0; PWDATA = 0;
#5;

if (start == 0)
    $display("START CLEAR PASS");
else
    $display("START CLEAR FAIL: start = %b", start);
```

This section is written inline rather than through the apb_write task because it needs to inspect start at two different moments: right after the write takes effect, and one cycle later. Writing CTRL bit 0 = 1 should pulse start high for exactly one clock cycle, since the DUT's write-phase logic sets start <= 1'b0 by default at the top of every clock and only overrides it to PWDATA [0] on a qualifying write.

What to expect:

“START PASS” immediately after the write, confirming start went high, followed by “START CLEAR PASS” one cycle later, confirming it automatically dropped back to 0 without any further APB action proving start behaves as the intended one clock pulse rather than a level that would re-trigger the SPI master on every subsequent clock.

1.10 RX_DATA Read

```
rx_data = 32'hdddd;  
apb_read(4'h4, 32'hdddd);
```

Since rx_data is driven directly by the testbench in this isolated test (standing in for the real SPI master), this simply confirms that reading address 0x4 passes the master's received word straight through to PRDATA with no modification.

What to expect:

“READ PASS: ADDR=4 DATA=dddd”.

1.11 STATUS Register — All Four BUSY/DONE Combinations

```
busy = 0; done = 0; apb_read(4'hC, 32'h0);  
busy = 1; done = 0; apb_read(4'hC, 32'h1);  
busy = 0; done = 1; apb_read(4'hC, 32'h2);  
busy = 1; done = 1; apb_read(4'hC, 32'h3);
```

STATUS packs busy into bit 0 and done into bit 1. Testing all four combinations (00, 01, 10, 11) exhaustively covers the 2-bit field rather than just spot-checking one case, which would leave bit-ordering mistakes (e.g. swapping busy and done) undetected.

What to expect:

Four consecutive lines, all READ PASS, with DATA reading 0, 1, 2 and 3 respectively confirming busy occupies bit 0 and done occupies bit 1 as documented.

1.12 Read-Only Register Protection

```
rx_data = 32'h5555_AAAA;  
apb_write(4'h4, 32'hFFFF_FFFF);  
#1;  
if (rx_data == 32'h5555_AAAA)  
    $display("RX_DATA READ-ONLY PASS");  
else  
    $display("RX_DATA READ-ONLY FAIL");  
  
busy = 1; done = 0;  
apb_write(4'hC, 32'hFFFF_FFFF);  
#1;  
if (busy == 1 && done == 0)  
    $display("STATUS READ-ONLY PASS");  
else  
    $display("STATUS READ-ONLY FAIL");
```

Both RX_DATA and STATUS are read-only from software's point of view — they are driven by the SPI master, not by the APB bus. These checks attempt to write all-ones to each address and then confirm the underlying signal was left completely untouched, proving the DUT's write-decode case statement has no accidental case for 0x4 or 0xC.

What to expect:

“RX_DATA READ-ONLY PASS” and “STATUS READ-ONLY PASS” — the write attempts should have no effect at all on rx_data, busy or done.

1.13 Invalid Address

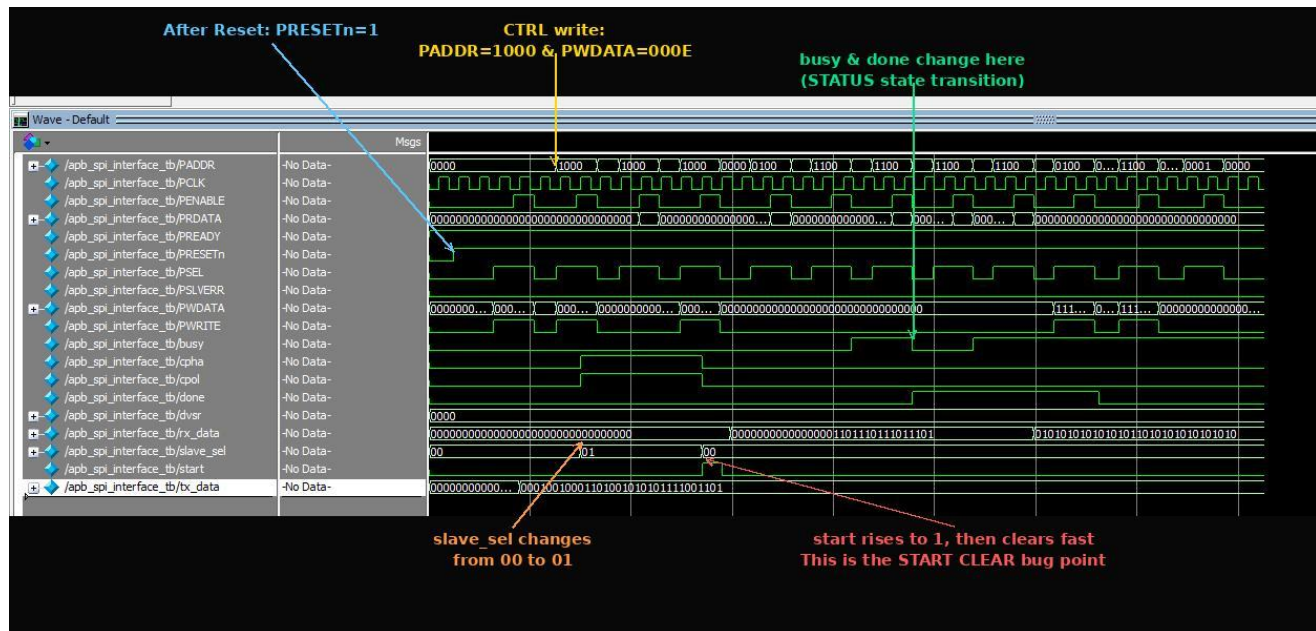
```
apb_read(4'h1, 32'h0);
```

Address 0x1 is not decoded by the read case statement, so it should fall into the default branch, which drives PRDATA to all zeros.

What to expect:

“READ PASS: ADDR=1 DATA=0”, confirming the register file fails safe (returns zero) rather than returning stale or undefined data for an unmapped address.

APB interface waveform:



2. SPI Master Testbench

Unlike the APB testbench, this one tests real protocol timing: it starts genuine SPI transactions and lets the master's own FSM generate SCLK and shift 32 bits, then checks the data it receives back against what it sent.

2.1 The Loopback Connection

```
wire miso = mosi;
```

Instead of connecting to a real SPI slave, MISO is tied directly back to the master's own MOSI output. Whatever bit the master shifts out is the exact same bit it reads back in on the next sample edge. This turns the testbench into a self-contained check of the master's shift-register and timing logic: if the master's internal transmit and receive shifting are both correct, data_out will end up exactly equal to data_in, regardless of which of the four SPI modes is used.

What to expect:

Because of the loopback, this test can validate the master's clock generation, CPOL/CPHA edge selection and bit-counting all in one shot, without needing the slave module at all — the slave module is verified separately in Section 4.

2.2 Clock Generation and Reset

```
always #5 clk = ~clk;

task reset_dut;
begin
    reset    = 0;
    start    = 0;
    cpol     = 0;
    cpha     = 0;
    data_in  = 32'h0;
    dvr      = 4'd2;
    slave_sel = 2'd0;
    #20;
    reset = 1;
    #20;
end
endtask
```

The system clock is 100 MHz (10 ns period), same as the APB testbench. reset_dut asserts the master's active-low reset for 20 ns two full clock periods before releasing it, which is comfortably longer than one clock edge and guarantees the FSM is cleanly forced into IDLE with cs fully de-asserted before any transaction begins.

2.3 The do_transaction Task

```
task do_transaction(input [31:0] din, input cpol_v, input cpha_v,
                   input [3:0] dvr_v, input [1:0] ssel);
    begin
        data_in    = din;
        cpol       = cpol_v;
        cpha       = cpha_v;
        dvr        = dvr_v;
        slave_sel   = ssel;
        start      = 1;
        #10;
        start      = 0;

        wait (done == 1'b1);
        #10;

        $display("din=%h cpol=%b cpha=%b dvr=%0d ssel=%0d dout=%h %s",
                din, cpol_v, cpha_v, dvr_v, ssel, data_out,
                (data_out === din) ? "PASS" : "FAIL");

        #20;
    end
endtask
```

This task is the whole testbench's engine: it loads a transmit word and a full set of mode/config settings, pulses start for one clock (matching how the real APB interface would pulse it), and then uses a Verilog wait (done == 1'b1) statement to block until the master's FSM finishes the entire 32-bit transaction and returns to IDLE by itself the testbench does not need to know how many clock cycles that takes. Once done, it compares data_out against the original data_in using a case-equality (===) check and prints a single self-checking PASS/FAIL line.

What to expect:

For a healthy design, data_out should equal data_in exactly, in every mode, because of the MISO/MOSI loopback any mismatch would point to a bug in the shift timing for that specific CPOL/CPHA combination or an off-by-one in the bit counter.

2.4 Test Sequence — All Four SPI Modes

```
do_transaction(32'h0001_0092, 1'b0, 1'b0, 4'd2, 2'd0); // CPOL0 CPHA0
do_transaction(32'h1234_5678, 1'b0, 1'b1, 4'd3, 2'd1); // CPOL0 CPHA1
do_transaction(32'h74a0_0980, 1'b1, 1'b0, 4'd1, 2'd2); // CPOL1 CPHA0
do_transaction(32'h0392_1733, 1'b1, 1'b1, 4'd0, 2'd3); // CPOL1 CPHA1
```

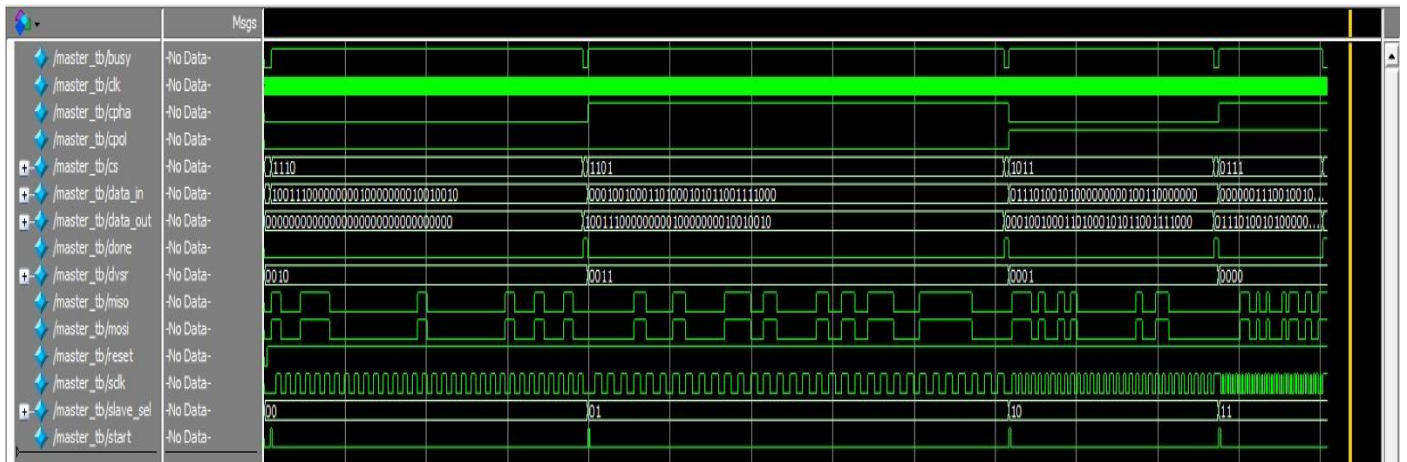
Four back-to-back transactions, each with a different 32-bit data pattern, a different CPOL/CPHA pair covering all four standard SPI modes, a different clock divider (2, 3, 1, 0) so the SCLK frequency differs each time, and a different slave_sel value (0–3) so all four chip-select outputs get exercised at least once.

What to expect:

Four PASS lines on the console, one per transaction. On the waveform (dumped to master_tb.vcd), you should see: sclk idling at the level set by cpol between transactions; the appropriate cs bit pulled low only for the duration of that transaction and returning to 4'b1111 afterward; mosi changing on the edge dictated by

cpha; and busy staying high for the entire 32-bit shift before dropping alongside a one-cycle done pulse at the end of each transaction.

SPI master waveform:



3. SPI Slave Testbench

This testbench drives the slave module and it plays the role of an external SPI master by generating its own SCLK and driving MOSI, so it can check the slave in isolation without need for the master module.

4.1 s0 Instantiation

```
slave s0 (.*);
```

The `.*` implicit port connection call by name that we were using during the course automatically connects every `s0` port to a testbench signal of the same name (`sclk`, `reset`, `mosi`, `miso`, `done`, `cs`), saving a full explicit port list and as we know that it requires every port name on the module to already exist as a matching signal in the testbench, which I already did here.

4.2 SPI Clock Generation

```
initial begin
    sclk = 1'b0;
    forever #5 sclk = ~sclk;
end
```

Since there is no master in this test, the testbench itself must generate a clean SCLK without the master need.

4.3 Driving 32 Bits onto MOSI

```
test_data = 32'hAB31_0092;
#20;
reset = 1'b1;
#10;

cs = 1'b0;

for (i = 31; i >= 0; i = i - 1) begin
    mosi = test_data[i];
    @(posedge sclk);
end

@(negedge sclk);
```

After releasing reset and asserting chip-select (active low, so $cs = 0$ selects the slave), the loop counts i from 31 down to 0, placing `test_data[i]` the MSB first onto MOSI and then waiting for the next rising edge of `sclk` before moving to the next bit.

This matches the slave module's logic, which shifts `rx_reg` left and inserts MOSI on every posedge `sclk` while `cs` is low, and it matches the master's MSB-first shifting convention, so the two modules agree on bit order.

What to expect:

After 32 rising edges, `rx_reg` inside the slave should hold exactly `32'hAB31_0092`, bit-for-bit identical to `test_data`.

NOTE: The extra `@(negedge sclk)` after the loop lets the slave's falling-edge block run once more so its done flag has a chance to update before the check below reads it.

4.4 Self-Check via Hierarchical Reference

```
if (done == 1'b1)
    $display("DONE TEST: PASS");
else
    $display("DONE TEST: FAIL");

if (s0.rx_reg == test_data)
    $display("RX TEST: PASS  RX = %h", s0.rx_reg);
else
    $display("RX TEST: FAIL  Expected = %h  Got = %h", test_data, s0.rx_reg);
```

Two independent checks confirm the transfer worked. The first reads the slave's done output port, which the `s0` call sets after counting exactly 32 falling edges of `sclk`. The second reaches directly inside the DUT using the hierarchical path `s0.rx_reg` to read the internal shift register itself, rather than relying only on the external `miso/done` ports — a white-box check that catches cases where the shift register holds the right data but the status/output logic is wrong, or vice versa.

What to expect:

“DONE TEST: PASS” followed by “RX TEST: PASS RX = ab310092”. If the bit ordering or the falling-edge bit counter had an off-by-one, RX TEST would fail while DONE TEST might still pass (or vice versa) which is exactly why both checks are kept separate rather than combined into one.

SPI Slave wave form:

